

```

•   set: function (newValue) {
•     var names = newValue.split(' ')
•     this.firstName = names[0]
•     this.lastName = names[names.length - 1]
•   } } }
• // ...

```

• Watchers

Although computed properties are more appropriate, there are times when you may require a custom watcher. This is why Vue provides a more inclusive way to react to data changes through the option “**watch**”. This is, in turn, more useful when needed to perform asynchronous or expensive operations. For Example:

```

• <div id="watch-example">
•   <p>
•     Ask a yes/no question:
•     <input v-model="question">
•   </p>
•   <p>{{ answer }}</p>
• </div>

```

In this case, the **watch** option would allow us to perform an asynchronous operation that would limit how often we perform that operation and set intermediary states until we reach a final answer. None of that would be possible with a computed property. In addition, you can also use **vm.\$watch API** instead of the **watch** option.

Conclusion

Computed properties are defined by stored properties, classes, structure, and enumerations, which do not allow storing a value. Instead of this, they provide a getter that further sets properties and values indirectly.

Computed properties are used to handle complex logic and operations. These are just like methods but have certain differences that make them convenient and useful for simple operations. The primary difference between methods and computed property is that we refer to it as property when talking about computed properties. In contrast, we call it a function in the case of methods. The in-template expressions are used for simple operations, as putting a lot of pressure, and heavy and complex logic in the templates make it difficult to maintain. **d**